

Assignment-1 (SOC'26 :- KernelMind)

24B1211, Dhruv Singla

- 1.1 (A) When hardware timer is interrupted , the process is executing so current state is running state and then cpu saves basic execution state before kernel interrupt handler starts which saves full context into PCB which is called context and process is called context switching so state changes to ready stage and OS scheduler chooses another process to be runned based on scheduling policy and dispatcher will load context of that new process from PCB and new process will start running .
- 1.1 (B) If no data is available then process must wait which means it should be blocked not terminated then when data arrives on hardware interrupt it moves to ready state then scheduler selects process so it moves to running state and the will terminate at last so correct sequence will be :
Running > blocked > ready > running > termination
- 1.1 (C) Zombie process remains there because the parent process has not yet collected in exit status . kernel keeps the entry so that parent can retrieve its exit code , termination info , and accounting data and it remains until parent calls wait() or waitpid() , which collects info and delete the zombie. And if parent exist earlier than zombie than zombie is adopted by init/system (PID1) which later calls wait () and cleans up the zombie automatically.
- 1.2 (A) Generally after n fork iterations we have (2^n) processes as evry existing process forks again if there are no process exits during the loop but in the code given in the question process A never executes the second fork but its only contrld by fork B instead so therefore only 3 processes are created in this case because we are calling the exit status in the loop and its being conditioned on the second fork to generate the third one
- 1.2 (B) Exact output to be printed by the code : C: g=110 x=105, B: g=10 x=5 , A: g=10 x=5 as process C modifies just the private copies of g and x but B and A retain their original values ; and the output order is fixed because of wait() .
The global variable doesn't propagate because after every fork(); each process gets its own virtual memory copy so modifications occurs independently .

1.3 (C) B has wait(null) in its execution so c must finish before B prints thus b before c is impossible and similarly A also waits with wait null so B must Finish Before A prints so only possible ordering is C -> B -> A. and all other combinations are impossible in this code.

1.4 (D)

- i. No, after fork(), parent and child don't really share the same heap object as the OS uses copy_on_write (COW) optimization which means both of them reference the same physical mem page but have only read-only access and when either of them writes then kernel creates a private copy of page for it so ultimately, each process has its own independent heap memory.
- ii. Use-after-free bug can arise if mem is shared between processes and one process frees the mem before the other accesses it so such bugs can cause crashes and corrupted data and can be detected using memory debuggers or address sanitizers.
- iii. In this case C becomes an orphan process which is being handed over to init/system which is PID1 and this mechanism is called reparenting and it is important to collect zombies and collect exit status and repeatedly doing so can exhaust the process table and pid space leading to system instability.

2.1 (A) i. In srtf, pre-emption will happen only when newly arrived process has smaller remaining time than current running one, at $t=0$ only p_1 is there so it runs then at $t=2$, p_2 comes with lower time so it preempts over p_1 then at $t=4$, $p_3(6) > p_2(2)$ so no pre-empt then at $t=6$, p_2 finishes and then p_4 is picked with minimum time of 2 then similarly on its completion at $t=8$, p_3 is chosen so only pre-emption occurs at $t=2$ when p_1 is pre-empted for p_2 and p_2 starts running

ii. yes, in srtf, p_1 can finish last as short jobs will repeatedly jump ahead due to its longer burst time despite arriving first. This is guaranteed when every arriving process has shorter time than p_1 and is impossible if no shorter jobs arrive after p_1 arrives so that will drive it to its completion earlier

(B) in non preemptive sjf, it produces lesser waiting time on avg because it always selects the shortest available burst at any given point on time such that shorter jobs won't have to wait for bigger jobs to be completed for their execution in our example fcfs will be heavily influenced by p_1 which arrives first

so main contributors are p2 and p4 with short jobs which avoid waiting in sjf behind the long execution of p1 so it reduces the convoy effect of fcfs .

(C) Completion Times

Process	Completion
P1	28
P2	15
P3	24
P4	14
P5	30

Turnaround Time

$$TAT = Completion - Arrival$$

Process	TAT
P1	28
P2	13
P3	20
P4	8
P5	22

Waiting Time

$$WT = TAT - Burst$$

Process	WT
P1	18
P2	9
P3	14
P4	6
P5	14
Waiting avg = 12.2 and turnaround avg = 18.2	

And this happens because rr interrupts jobs evn when completion is near because it prioritizes fairness over optimality .

(D) Waiting Times

Process	Waiting Time
P1	0
P2	10
P3	12
P4	4
P5	14

Average WT = 8 ms

Turnaround Times

Process	Turnaround Time
P1	10
P2	14
P3	18
P4	6
P5	22

Average TAT = 14 ms

2.2(a) Given, $b_1 \leq b_2 \leq b_3$

For non-preemptive SJF, the order is (b_1, b_2, b_3)

So, waiting times are $w_1 = 0$, $w_2 = b_1$, $w_3 = b_1 + b_2$

$$\text{Avg. waiting time} = \frac{2b_1 + b_2}{3}$$

Now, consider for any other schedule when a longer job appears before a shorter one then,
 $(\dots, b_j, b_i, \dots)$

$b_j > b_i$ then $w_i = b_j$

And if b_i and b_j are swapped then,
 $w_i = b_i$

And since $b_j > b_i$, total waiting time either decreases or remains unchanged and if any schedule contains such inversion of at least a pair then it would perform worse than SJF.
And if we keep on swapping adjacent pairs, we will get a sequence exactly same to that of an SJF.
Hence, SJF minimizes avg. waiting time.

$$(b) i. \text{ Total response time for SJF} = \frac{n(0 + 1 + 2 + \dots + (k-1)) + ks}{(k+1)}$$

$$= \frac{nk}{2}$$

$\Rightarrow$ Response time in round-robin $\Rightarrow$

1st process

2nd

3rd

$\vdots$

last

time

0

1

2q

$\vdots$

kq

Date.....

$$\therefore \text{Avg RR response time} = \frac{g_k}{2}$$

and given that $g < d \Rightarrow \frac{g_k}{2} < \frac{g_k}{2}$

$\therefore$ RR gives better response time

$\Rightarrow$ Intuitively, I can say this happens because RR optimizes responsiveness over completion and waiting, as all of the jobs gets CPU quickly.

3.1(A)	Process	Wait time	Turnaround time
	P_1	0	12
	P_2	12	15
	P_3	15	22
	P_4	22	24
	P_5	24	30

Since, all of them arrive at $T=0$

$$\therefore \text{Wait time} = \text{Start time}$$

$$\text{Turnaround time} = \text{finish time}$$

(B) Makespan = $30 - 0 = 30$ ms

$$(C) JFI = \frac{\sum x_i}{n \sum x_i^2} = \frac{(0+12+15+22+24)}{5(0+144+225+484+576)}$$

$$\approx 0.716$$

(D) Using SJF, shorter jobs are scheduled earlier while longer jobs wait longer. which reduces avg. waiting time but increases inequality. while FIFS, distributes it more uniformly according to arrival order.
 $J_{SJF} < J_{FIFS}$

3.2(G) Scheduler beta most likely belongs to a SJF/SRTF family because it produces extremely low wait time while producing very large lag latency because short jobs complete quickly while long jobs suffer starvation.

3.2 (B) makespan can be minimized by maximizing cpu utilization and throughput ensuring overall workload completes as quickly as possible. However, this does not necessarily minimize the waiting experienced by individual jobs. Scheduler Gamma likely prioritizes keeping the processor continuously busy and efficiently draining the workload, which reduces total completion time but allows some jobs to wait longer, increasing mean wait time. Makespan is often not the best metric for interactive workloads because users care more about latency and responsiveness than total throughput. An interactive application is sensitive to tail delays (such as P90/P99 wait times), since even a few long delays can noticeably degrade user experience.

3.2(C) i. the best choice is Gamma because the workload prioritizes throughput and total completion time

ii. the best choice is Delta because interactive systems are highly sensitive to tail latency